

Manual Técnico

Sistema de Registro de Proyectos — LUBConsultores

Caso 8

Versión: 1.0
Fecha: Marzo 2026
Elaborado por: Daniel Casasola

Tabla de Contenidos

- 1. Descripción del Sistema
- 2. Stack Tecnológico
- 3. Arquitectura del Proyecto
- 4. Estructura de Carpetas
- 5. Requisitos Previos
- 6. Instalación y Configuración
- 7. Variables de Entorno
- 8. Modelo de Base de Datos
- 9. Endpoints de la API
- 10. Comandos Útiles

1. Descripción del Sistema

El Sistema de Registro de Proyectos es una aplicación web desarrollada para **LUBConsultores**, una firma de consultoría que necesita llevar el control y seguimiento de los proyectos que desarrollan para sus clientes.

El sistema permite:

- Registrar proyectos con su información general (número único, descripción, fechas, monto, cliente y consultor asignado).
- Dar seguimiento periódico a cada proyecto mediante una bitácora de estados (Abierto, Suspendido, Finalizado), permitiendo que un proyecto tenga el mismo estado en múltiples ocasiones sin perder el historial.
- Gestionar el historial de montos de cada proyecto de forma acumulativa.
- Administrar catálogos base: roles, estados, prioridades y empresas.
- Registrar clientes y consultores.
- Buscar y generar reportes exportables en formato CSV.
- Visualizar métricas y alertas en un dashboard general.

2. Stack Tecnológico

Capa	Tecnología	Versión
Frontend	Vue 3	^3.5.x
Build Tool	Vite	^7.x
Estilos	Tailwind CSS	^4.x
Estado	Pinia	^3.x
Routing	Vue Router	^4.x

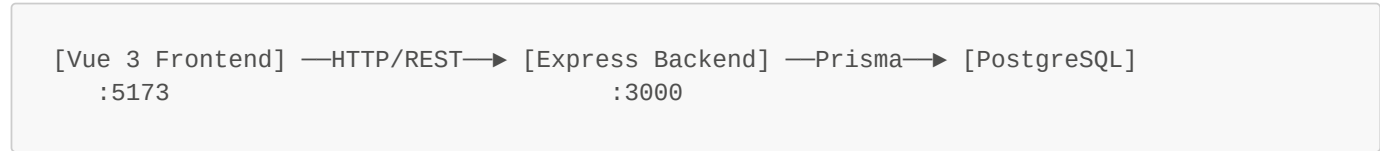
Capa	Tecnología	Versión
HTTP Client	Axios	^1.x
Gráficas	Chart.js	^4.x
Backend	Express	^4.x
Lenguaje	TypeScript	~5.9.x
ORM	Prisma	^7.x
Base de Datos	PostgreSQL	16+
Entorno	Node.js	20+
Gestor de paquetes	npm workspaces	—

3. Arquitectura del Proyecto

El proyecto está organizado como un **monorepo** utilizando npm workspaces, con dos paquetes principales: **frontend** y **backend**.

```
registro-proyectos-webapp/  
├── packages/  
│   ├── frontend/    ← Vue 3 + Vite + Tailwind  
│   └── backend/     ← Express + TypeScript + Prisma  
├── docs/            ← Documentación  
├── package.json     ← Raíz del monorepo  
└── README.md
```

El frontend se comunica con el backend a través de una API REST. El backend se conecta a PostgreSQL usando Prisma como ORM.



4. Estructura de Carpetas

Frontend (**packages/frontend/**)

```
src/  
├── assets/           ← Recursos estáticos  
├── components/  
│   └── ui/          ← Componentes reutilizables  
│       ├── ModalForm.vue  
│       ├── CatalogoTable.vue  
│       └── SearchableSelect.vue  
├── layouts/  
│   └── MainLayout.vue ← Layout principal con sidebar  
├── router/  
│   └── index.ts      ← Definición de rutas  
├── services/        ← Llamadas a la API  
│   ├── api.ts  
│   └── catalogo.service.ts
```

```

├── cliente.service.ts
├── consultor.service.ts
├── proyecto.service.ts
├── bitacora.service.ts
├── logMonto.service.ts
├── usuario.service.ts
├── stores/                ← Estado global con Pinia
├──   ├── catalogo.store.ts
├──   └── proyecto.store.ts
├── types/
├──   └── index.ts          ← Tipos TypeScript
├── views/                 ← Vistas principales
├──   ├── DashboardView.vue
├──   ├── ProyectosView.vue
├──   ├── ProyectoDetalleView.vue
├──   ├── ClientesView.vue
├──   ├── ConsultoresView.vue
├──   ├── CatalogosView.vue
├──   └── BusquedaReportesView.vue
├── App.vue
├── main.ts
└── style.css

```

Backend ([packages/backend/](#))

```

src/
├── controllers/           ← Reciben la request y responden
├──   ├── catalogo.controller.ts
├──   ├── cliente.controller.ts
├──   ├── consultor.controller.ts
├──   ├── proyecto.controller.ts
├──   ├── bitacora.controller.ts
├──   ├── logMonto.controller.ts
├──   └── usuario.controller.ts
├── services/             ← Lógica de negocio y consultas Prisma
├──   ├── catalogo.service.ts
├──   ├── cliente.service.ts
├──   ├── consultor.service.ts
├──   ├── proyecto.service.ts
├──   ├── bitacora.service.ts
├──   ├── logMonto.service.ts
├──   └── usuario.service.ts
├── routes/              ← Definición de rutas
├──   ├── catalogo.routes.ts
├──   ├── cliente.routes.ts
├──   ├── consultor.routes.ts
├──   ├── proyecto.routes.ts
├──   ├── bitacora.routes.ts
├──   ├── logMonto.routes.ts
├──   └── usuario.routes.ts
├── db/
├──   └── prisma.ts        ← Instancia del cliente Prisma
├── generated/
├──   └── prisma/          ← Cliente generado por Prisma
├── index.ts              ← Punto de entrada del servidor
└── prisma/
├──   ├── schema.prisma   ← Definición del esquema
├──   └── migrations/     ← Historial de migraciones

```

5. Requisitos Previos

Antes de instalar el proyecto asegúrate de tener:

- **Node.js** 20 o superior
- **npm** 10 o superior
- **PostgreSQL** 16 o superior
- **Git**

Para verificar las versiones instaladas:

```
node --version
npm --version
psql --version
git --version
```

6. Instalación y Configuración

1. Clonar el repositorio

```
git clone https://github.com/tuusuario/registro-proyectos-webapp.git
cd registro-proyectos-webapp
```

2. Instalar dependencias

Desde la raíz del monorepo:

```
npm install
```

3. Configurar la base de datos

Iniciar PostgreSQL (en Arch Linux):

```
sudo systemctl start postgresql
```

Crear la base de datos:

```
sudo -u postgres psql
```

```
CREATE DATABASE "registro_proyectos_db" WITH ENCODING = 'UTF8';
\q
```

4. Configurar variables de entorno

Crear el archivo `.env` en `packages/backend/`:

```
cp packages/backend/.env.example packages/backend/.env
```

Editar el archivo con tus credenciales de PostgreSQL.

Crear el archivo `.env` en `packages/frontend/`:

```
cp packages/frontend/.env.example packages/frontend/.env
```

5. Ejecutar migraciones de Prisma

```
cd packages/backend
npx prisma migrate dev
npx prisma generate
```

6. Crear usuario inicial

```
curl -X POST http://localhost:3000/api/usuarios \
  -H "Content-Type: application/json" \
  -d
'{"nombre": "Admin", "apellido": "Sistema", "correo": "admin@lub.com", "contrasena": "admin123", "id_rol": 1}'
```

7. Iniciar el proyecto

Backend (en `packages/backend/`):

```
npm run dev
```

Frontend (en `packages/frontend/`):

```
npm run dev
```

El frontend estará disponible en `http://localhost:5173` y el backend en `http://localhost:3000`.

7. Variables de Entorno

Backend (`packages/backend/.env`)

Variable	Descripción	Ejemplo
<code>DATABASE_URL</code>	URL de conexión a PostgreSQL	<code>postgresql://postgres:password@localhost:5432/registro_proyectos_db</code>

Frontend (`packages/frontend/.env`)

Variable	Descripción	Ejemplo
VITE_API_URL	URL base de la API del backend	http://localhost:3000/api

8. Modelo de Base de Datos

Tablas

rol

Campo	Tipo	Descripción
id_rol	INTEGER (PK)	Identificador único
nombre	VARCHAR(50)	Nombre del rol

usuarios

Campo	Tipo	Descripción
id_usuario	INTEGER (PK)	Identificador único
nombre	VARCHAR(50)	Nombre del usuario
apellido	VARCHAR(50)	Apellido del usuario
correo	VARCHAR(100) UNIQUE	Correo electrónico
contrasena	VARCHAR(255)	Contraseña hasheada
id_rol	INT (FK)	Referencia a rol
activo	BOOLEAN	Estado del usuario

empresa

Campo	Tipo	Descripción
id_empresa	INTEGER (PK)	Identificador único
nombre	VARCHAR(100)	Nombre de la empresa
direccion	VARCHAR(150)	Dirección física

estado_cliente

Campo	Tipo	Descripción
id_estado_cliente	INTEGER (PK)	Identificador único
estado	VARCHAR(50)	Nombre del estado

clientes

Campo	Tipo	Descripción
id_cliente	INTEGER (PK)	Identificador único
nombre	VARCHAR(100)	Nombre del cliente
apellido	VARCHAR(100)	Apellido del cliente

Campo	Tipo	Descripción
correo	VARCHAR(100) UNIQUE	Correo electrónico
telefono	VARCHAR(20)	Teléfono de contacto
id_empresa	INT (FK)	Referencia a empresa
id_estado_cliente	INT (FK)	Referencia a estado_cliente

consultores

Campo	Tipo	Descripción
id_consultor	INTEGER (PK)	Identificador único
nombre	VARCHAR(100)	Nombre completo
telefono	VARCHAR(20)	Teléfono
correo	VARCHAR(100) UNIQUE	Correo electrónico
activo	BOOLEAN	Estado del consultor
id_usuario	INT (FK, opcional)	Referencia a usuarios

estado_proyecto

Campo	Tipo	Descripción
id_estado_proyecto	INTEGER (PK)	Identificador único
estado	VARCHAR(50)	Nombre del estado

proyectos

Campo	Tipo	Descripción
id_proyecto	INTEGER (PK)	Identificador único
no_proyecto	VARCHAR(20) UNIQUE	Número único del proyecto
nombre	VARCHAR(100)	Nombre del proyecto
descripcion	TEXT	Descripción detallada
fecha_inicio	DATE	Fecha de inicio
fecha_fin	DATE	Fecha de finalización
id_cliente	INT (FK)	Referencia a clientes
id_consultor	INT (FK)	Referencia a consultores
id_estado_proyecto	INT (FK)	Referencia a estado_proyecto

prioridad

Campo	Tipo	Descripción
id_prioridad	INTEGER (PK)	Identificador único
nombre_prioridad	VARCHAR(50)	Nombre de la prioridad

bitacora

Campo	Tipo	Descripción
id_bitacora	INTEGER (PK)	Identificador único
fecha	DATE	Fecha del seguimiento
observaciones	TEXT	Descripción del seguimiento
id_estado_proyecto	INT (FK)	Estado registrado
id_proyecto	INT (FK)	Referencia a proyectos
id_usuario	INT (FK)	Usuario que registró
id_prioridad	INT (FK)	Prioridad del seguimiento

log_montos_proyectos

Campo	Tipo	Descripción
id_log_montos	INTEGER (PK)	Identificador único
monto	NUMERIC(12,2)	Monto registrado
observaciones	TEXT	Motivo del cambio
fecha	DATE	Fecha del registro
id_proyecto	INT (FK)	Referencia a proyectos
id_usuario	INT (FK)	Usuario que registró

Reglas de negocio importantes

- El campo **no_proyecto** es único — no pueden existir dos proyectos con el mismo número.
- La bitácora es acumulativa — un proyecto puede tener el mismo estado registrado múltiples veces sin que se borre el historial anterior.
- El monto del proyecto es histórico — cada cambio de monto se registra en **log_montos_proyectos**. El monto actual es el registro más reciente ordenado por fecha descendente.
- El estado actual del proyecto se actualiza automáticamente cada vez que se agrega un registro a la bitácora.

9. Endpoints de la API

Base URL: <http://localhost:3000/api>

Catálogos

Método	Endpoint	Descripción
GET	/roles	Obtener todos los roles
POST	/roles	Crear rol
PUT	/roles/:id	Actualizar rol
DELETE	/roles/:id	Eliminar rol
GET	/estados-cliente	Obtener estados de cliente
POST	/estados-cliente	Crear estado de cliente

Método	Endpoint	Descripción
PUT	/estados-cliente/:id	Actualizar estado de cliente
DELETE	/estados-cliente/:id	Eliminar estado de cliente
GET	/estados-proyecto	Obtener estados de proyecto
POST	/estados-proyecto	Crear estado de proyecto
PUT	/estados-proyecto/:id	Actualizar estado de proyecto
DELETE	/estados-proyecto/:id	Eliminar estado de proyecto
GET	/prioridades	Obtener prioridades
POST	/prioridades	Crear prioridad
PUT	/prioridades/:id	Actualizar prioridad
DELETE	/prioridades/:id	Eliminar prioridad
GET	/empresas	Obtener empresas
POST	/empresas	Crear empresa
PUT	/empresas/:id	Actualizar empresa
DELETE	/empresas/:id	Eliminar empresa

Usuarios

Método	Endpoint	Descripción
GET	/usuarios	Obtener todos los usuarios
GET	/usuarios/:id	Obtener usuario por ID
POST	/usuarios	Crear usuario
PUT	/usuarios/:id	Actualizar usuario
DELETE	/usuarios/:id	Eliminar usuario

Consultores

Método	Endpoint	Descripción
GET	/consultores	Obtener todos los consultores
GET	/consultores/:id	Obtener consultor por ID
POST	/consultores	Crear consultor
PUT	/consultores/:id	Actualizar consultor
DELETE	/consultores/:id	Eliminar consultor

Clientes

Método	Endpoint	Descripción
GET	/clientes	Obtener todos los clientes
GET	/clientes/:id	Obtener cliente por ID

Método	Endpoint	Descripción
POST	/clientes	Crear cliente
PUT	/clientes/:id	Actualizar cliente
DELETE	/clientes/:id	Eliminar cliente

Proyectos

Método	Endpoint	Descripción
GET	/proyectos	Obtener todos los proyectos
GET	/proyectos/:id	Obtener proyecto por ID
GET	/proyectos/buscar/:no_proyecto	Buscar por número de proyecto
POST	/proyectos	Crear proyecto
PUT	/proyectos/:id	Actualizar proyecto
DELETE	/proyectos/:id	Eliminar proyecto

Bitácora

Método	Endpoint	Descripción
GET	/bitacora/proyecto/:id_proyecto	Obtener bitácora de un proyecto
GET	/bitacora/:id	Obtener registro por ID
POST	/bitacora	Crear registro de bitácora
PUT	/bitacora/:id	Actualizar registro
DELETE	/bitacora/:id	Eliminar registro

Log de Montos

Método	Endpoint	Descripción
GET	/log-montos/proyecto/:id_proyecto	Historial de montos de un proyecto
GET	/log-montos/proyecto/:id_proyecto/ultimo	Último monto registrado
POST	/log-montos	Registrar nuevo monto
DELETE	/log-montos/:id	Eliminar registro de monto

10. Comandos Útiles

Backend

```
# Iniciar en modo desarrollo
npm run dev

# Compilar TypeScript
npm run build

# Iniciar en producción
npm run start
```

```
# Ejecutar migraciones
npx prisma migrate dev --name nombre_migracion

# Generar cliente Prisma
npx prisma generate

# Ver base de datos visualmente
npx prisma studio

# Resetear base de datos (cuidado: borra todos los datos)
npx prisma migrate reset
```

Frontend

```
# Iniciar en modo desarrollo
npm run dev

# Compilar para producción
npm run build

# Vista previa del build
npm run preview
```

PostgreSQL

```
# Iniciar PostgreSQL (Arch Linux)
sudo systemctl start postgresql

# Detener PostgreSQL
sudo systemctl stop postgresql

# Habilitar inicio automático
sudo systemctl enable postgresql

# Conectarse a la base de datos
sudo -u postgres psql -d registro_proyectos_db

# Ver tablas
\dt

# Salir de psql
\q
```

Git

```
# Ver estado del repositorio
git status

# Agregar todos los cambios
git add .

# Crear commit
git commit -m "tipo: descripción del cambio"
```

```
# Subir cambios a GitHub  
git push origin main
```